

AWS DevOps Project Report

Production-like Application Deployment on AWS Free Tier

Prepared By: RavindraNadh Sharma

Table of Contents

- 1. Abstract
- 2. Project Objective
- 3. Technologies Used
- 4. AWS Services Used
- 5. Project Architecture
- 6. Prerequisites
- 7. Implementation Steps
- 8. Jenkins CI/CD Pipeline
- 9. Docker Deployment
- 10. Application Deployment
- 11. CloudWatch Monitoring
- 12. Amazon S3 Backup
- 13. Amazon SNS Notifications
- 14. Security Best Practices
- 15. Challenges & Solutions
- 16. Project Workflow
- 17. Results
- 18. Future Enhancements
- 19. Conclusion
- 20. References

1. Abstract

This project demonstrates the complete deployment of a production-like web application using AWS and DevOps tools.

The application source code is maintained in GitHub, automatically deployed through Jenkins, containerized using Docker, hosted on Amazon EC2, monitored using CloudWatch, backed up to Amazon S3 and integrated with Amazon SNS for notifications.

The project focuses on Infrastructure, Automation, Monitoring, Security and Continuous Integration/Continuous Deployment.

This project demonstrates the complete deployment of a production-like web application using AWS and DevOps tools.

The application source code is maintained in GitHub, automatically deployed through Jenkins, containerized using Docker, hosted on Amazon EC2, monitored using CloudWatch, backed up to Amazon S3 and integrated with Amazon SNS for notifications.

The project focuses on Infrastructure, Automation, Monitoring, Security and Continuous Integration/Continuous Deployment.

2. Project Objective

- Build a production-like environment.
 - Automate deployment using Jenkins.
 - Deploy Docker containers on EC2.
 - Store backups in Amazon S3.
 - Monitor resources using CloudWatch.
 - Configure notifications using SNS.
-
- Build a production-like environment.
 - Automate deployment using Jenkins.
 - Deploy Docker containers on EC2.
 - Store backups in Amazon S3.
 - Monitor resources using CloudWatch.
 - Configure notifications using SNS.

3. Technologies Used

Linux, Git, GitHub, Docker, Jenkins, Python Flask, AWS EC2, IAM, VPC, S3, CloudWatch and SNS.

Linux, Git, GitHub, Docker, Jenkins, Python Flask, AWS EC2, IAM, VPC, S3, CloudWatch and SNS.

4. AWS Services Used

Amazon EC2 provides compute resources.

Amazon VPC provides network isolation.

IAM controls permissions.

CloudWatch provides monitoring.

S3 stores backups.

SNS sends notifications.

Amazon EC2 provides compute resources.

Amazon VPC provides network isolation.

IAM controls permissions.

CloudWatch provides monitoring.

S3 stores backups.

SNS sends notifications.

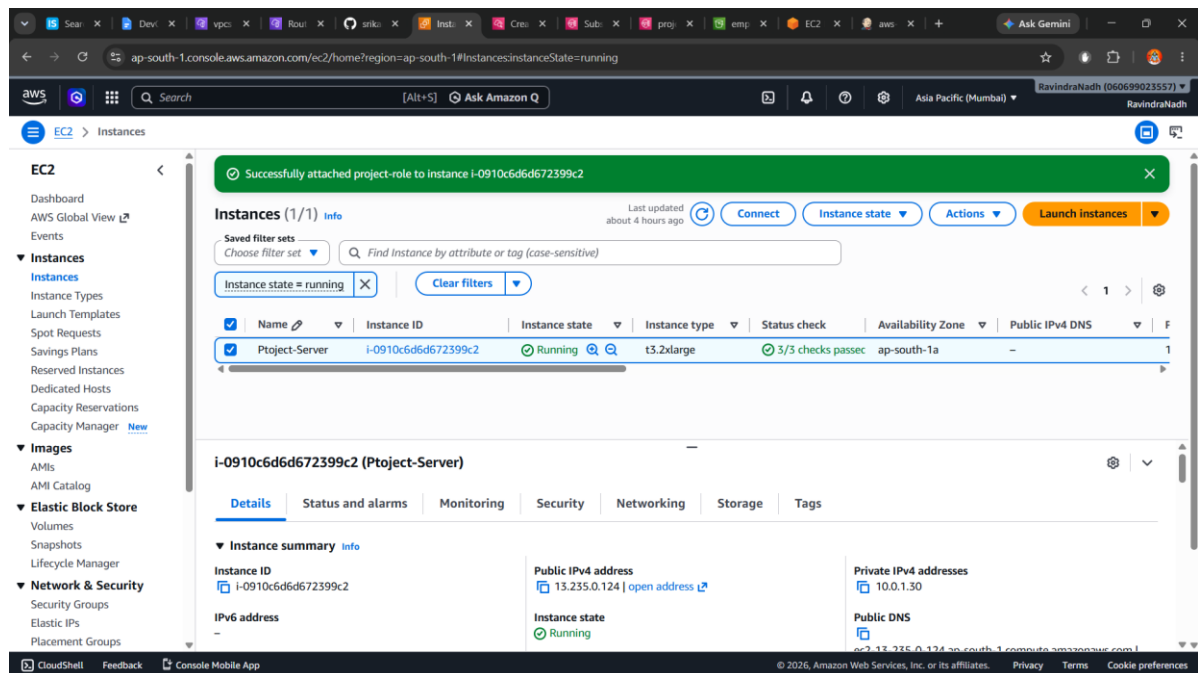


Figure: 4. AWS Services Used

5. Project Architecture

Developer → GitHub → Jenkins → Docker Build → Amazon EC2 → Flask Application

↓
CloudWatch / S3 / SNS

Developer → GitHub → Jenkins → Docker Build → Amazon EC2 → Flask Application

↓
CloudWatch / S3 / SNS

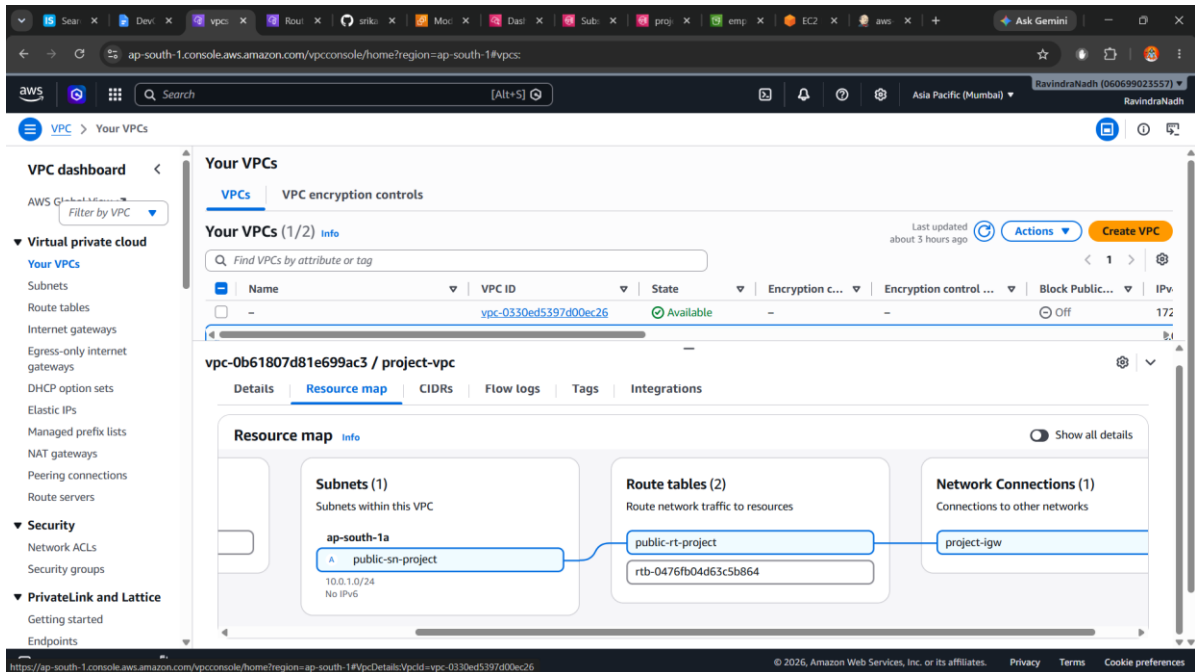


Figure: 5. Project Architecture

6. Prerequisites

AWS Account, EC2 instance, IAM Role, Docker installed, Jenkins installed, Git installed, Python Flask application.

AWS Account, EC2 instance, IAM Role, Docker installed, Jenkins installed, Git installed, Python Flask application.

7. Implementation Steps

1. Launch EC2.
2. Configure Security Groups.
3. Install Docker and Jenkins.
4. Clone GitHub repository.
5. Create Dockerfile.
6. Create Jenkinsfile.
7. Build Docker image.
8. Run Docker container.
9. Verify application.
10. Configure monitoring and backup.

1. Launch EC2.
2. Configure Security Groups.
3. Install Docker and Jenkins.
4. Clone GitHub repository.
5. Create Dockerfile.
6. Create Jenkinsfile.
7. Build Docker image.
8. Run Docker container.
9. Verify application.
10. Configure monitoring and backup.

8. Jenkins CI/CD Pipeline

Pipeline stages:

- Checkout source code
- Build Docker image
- Stop existing container
- Deploy latest container

Each code change triggers automated deployment.

Pipeline stages:

- Checkout source code
- Build Docker image
- Stop existing container
- Deploy latest container

Each code change triggers automated deployment.

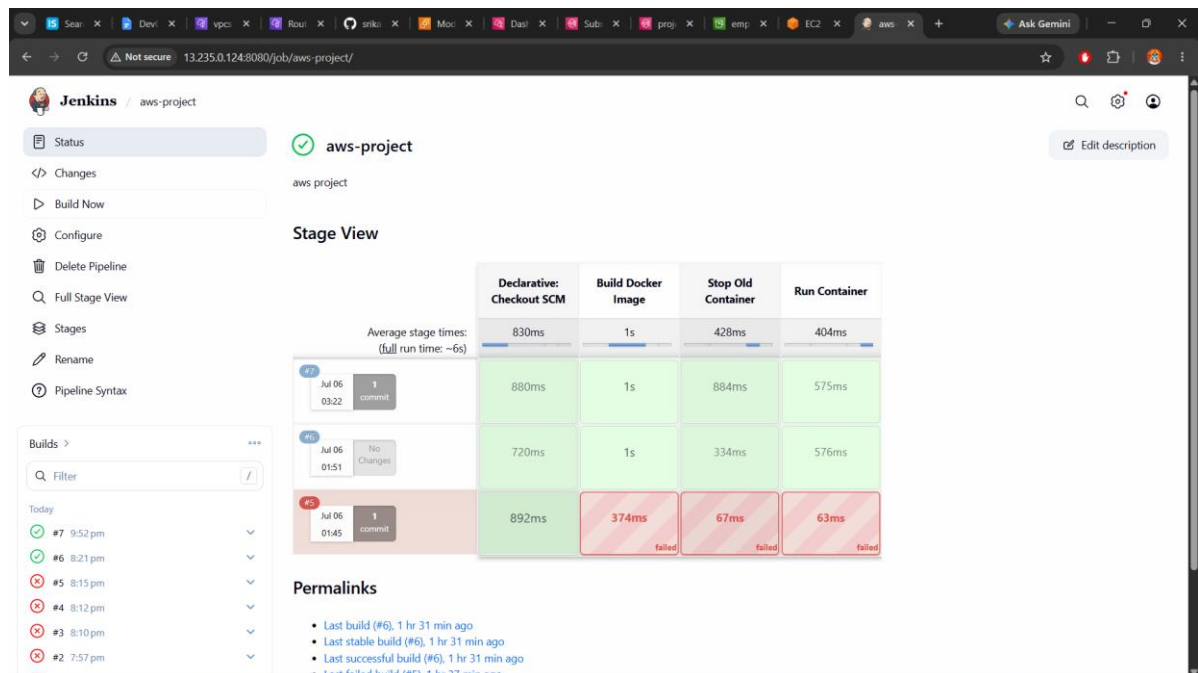


Figure: 8. Jenkins CI/CD Pipeline

9. Docker Deployment

Docker packages the application and its dependencies.

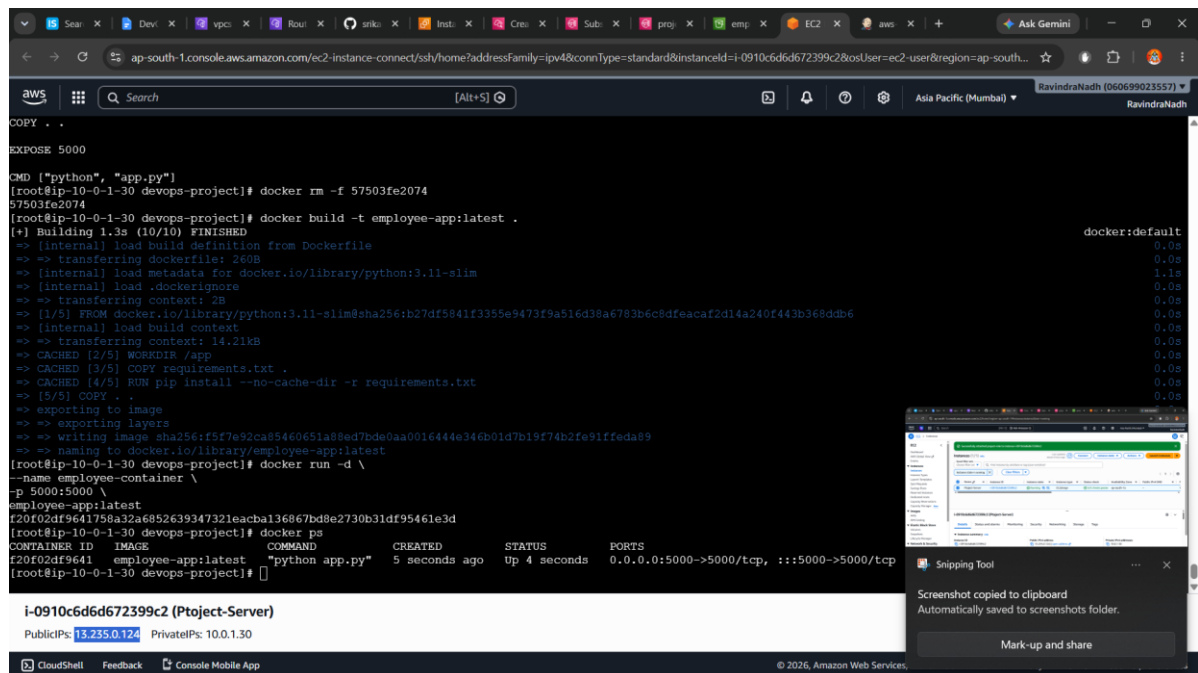
Benefits:

- Portability
- Consistent environment
- Fast deployment
- Easy rollback

Docker packages the application and its dependencies.

Benefits:

- Portability
- Consistent environment
- Fast deployment
- Easy rollback



The screenshot shows an AWS CloudShell terminal window with the following commands and output:

```
COPY . .
EXPOSE 5000

CMD ["python", "app.py"]
[root@ip-10-0-1-30 devops-project]# docker rm -f 57503fe2074
57503fe2074
[root@ip-10-0-1-30 devops-project]# docker build -t employee-app:latest .
[+] Building 1.3s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 260B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:b27df5841f335e9473f9a516d38a6783b6c8dfecaf2d14a240f443b368ddb6
=> [internal] load build context
=> => transferring context: 14.21kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> => exporting image
=> => exporting layers
=> => writing image sha256:f547e92ca85460651a88ed7bde0aa0016444e346b01d7b19f74b2fe91ffeda89
=> => naming to docker.io/library/employee-app:latest
[root@ip-10-0-1-30 devops-project]# docker run -d \
--name employee-container \
-p 5000:5000 \
employee-app:latest
f20f02df9641758a32a6852639347321eacba136867bd8e2730b31df95461e3d
[root@ip-10-0-1-30 devops-project]# docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS
f20f02df9641   employee-app:latest "python app.py"         5 seconds ago Up 4 seconds  0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
[root@ip-10-0-1-30 devops-project]#
```

At the bottom of the terminal, the instance information is displayed:

```
i-0910c6d6d72399c2 (Ptoject-Server)
PublicIPs: 13.235.0.124 PrivateIPs: 10.0.1.30
```

A Snipping Tool window is overlaid on the right side of the terminal, showing a screenshot of the terminal output. The tool has a message: "Screenshot copied to clipboard. Automatically saved to screenshots folder." and a "Mark-up and share" button.

Figure: 9. Docker Deployment

10. Application Deployment

The Flask application is exposed on port 5000 and accessed through the EC2 public IP.

The Flask application is exposed on port 5000 and accessed through the EC2 public IP.

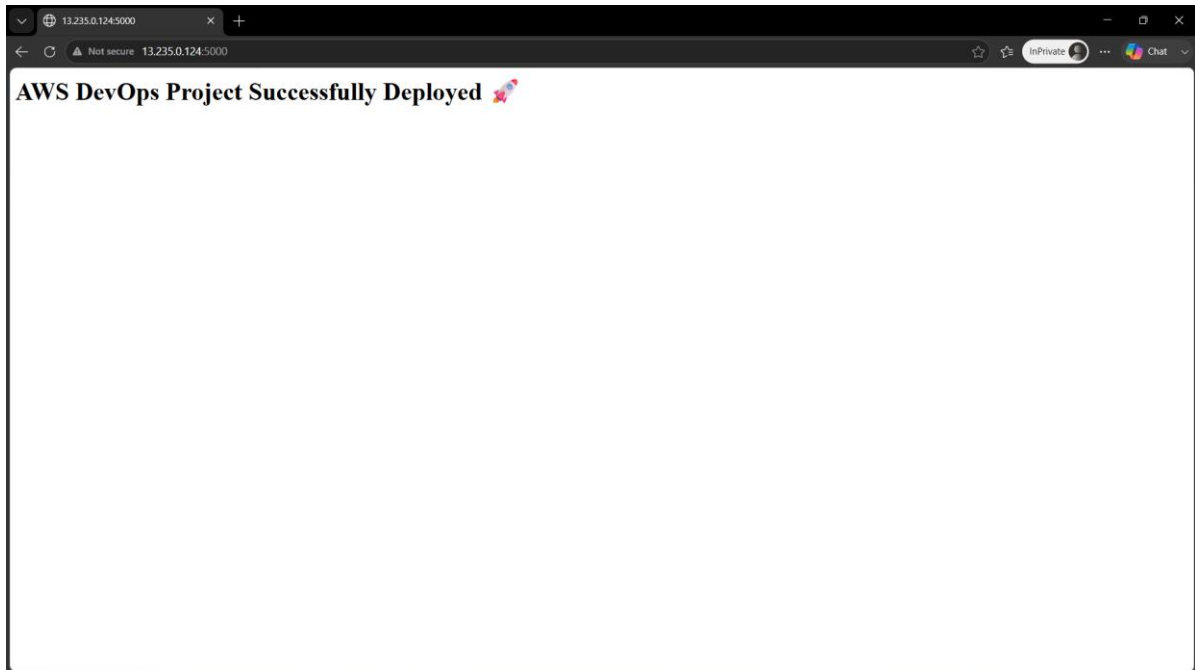


Figure: 10. Application Deployment

11. CloudWatch Monitoring

CloudWatch dashboard displays CPU utilization, Network In/Out, Memory Usage and Disk Usage.

CloudWatch dashboard displays CPU utilization, Network In/Out, Memory Usage and Disk Usage.

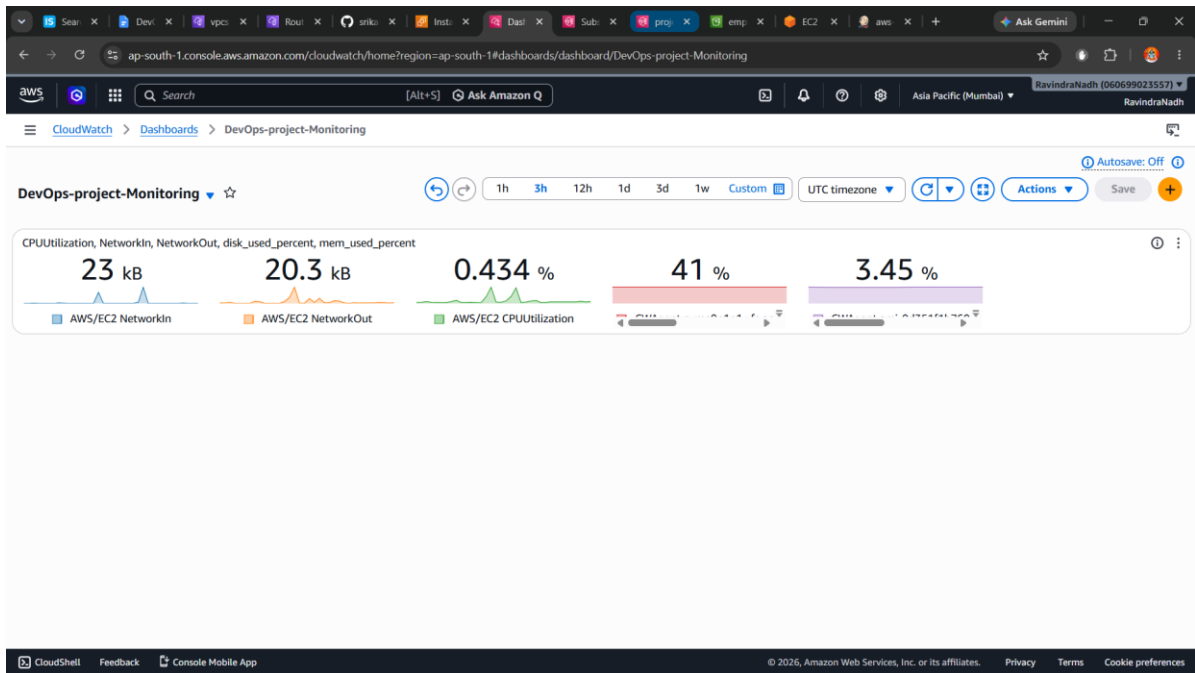


Figure: 11. CloudWatch Monitoring

12. Amazon S3 Backup

Application source code, Dockerfile, Jenkinsfile and requirements.txt are backed up into Amazon S3.

Application source code, Dockerfile, Jenkinsfile and requirements.txt are backed up into Amazon S3.

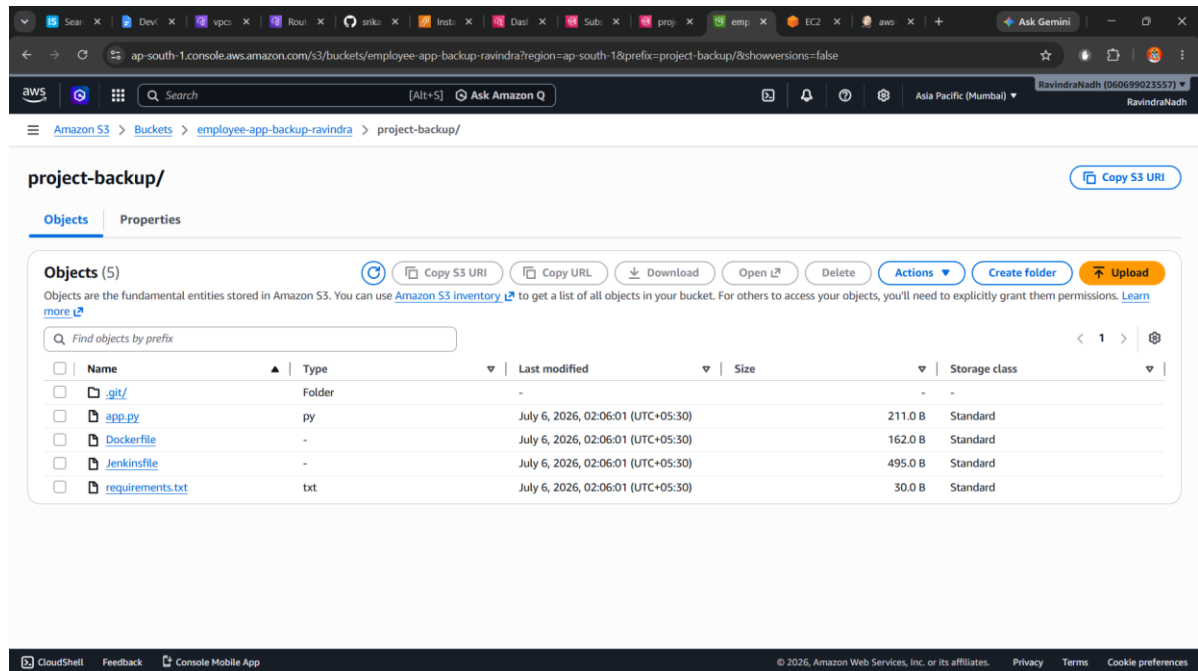


Figure: 12. Amazon S3 Backup

13. Amazon SNS Notifications

SNS topic is configured with email subscription to deliver infrastructure alerts.

SNS topic is configured with email subscription to deliver infrastructure alerts.

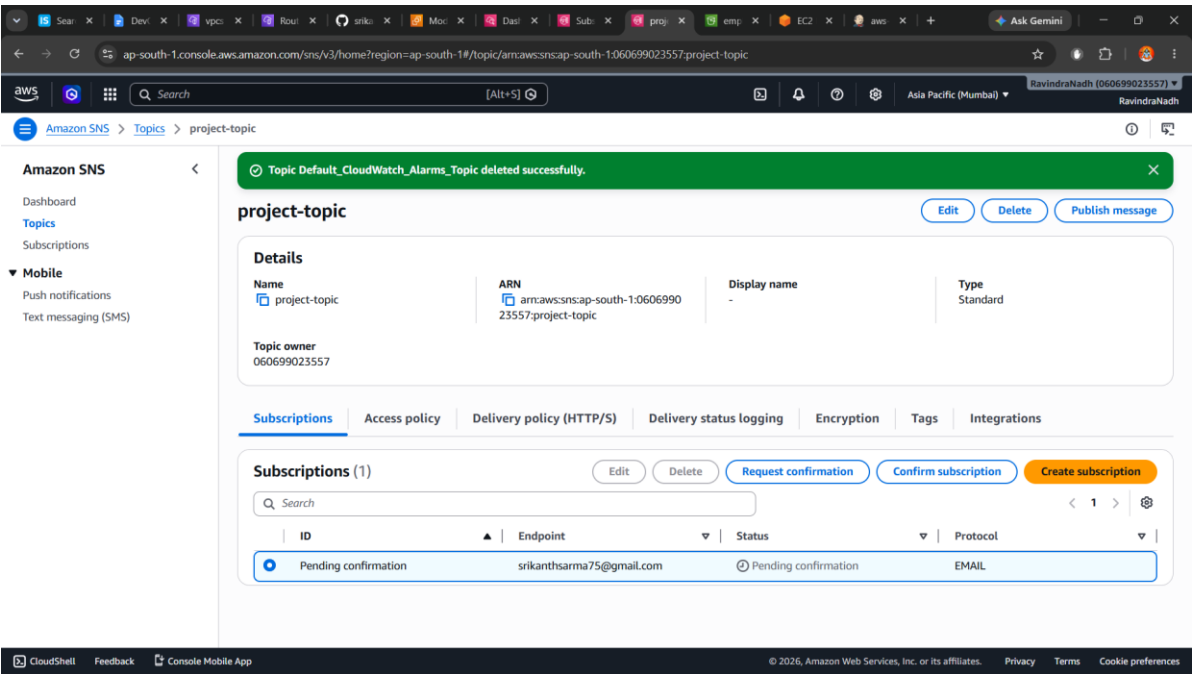


Figure: 13. Amazon SNS Notifications

14. Security Best Practices

- IAM Role attached to EC2
 - Least privilege principle
 - Security Groups
 - Docker container isolation
 - Backup strategy
-
- IAM Role attached to EC2
 - Least privilege principle
 - Security Groups
 - Docker container isolation
 - Backup strategy

15. Challenges & Solutions

Docker build issues, Jenkins pipeline failures and IAM permission issues were resolved through troubleshooting, proper pipeline configuration and IAM role assignment.

Docker build issues, Jenkins pipeline failures and IAM permission issues were resolved through troubleshooting, proper pipeline configuration and IAM role assignment.

16. Project Workflow

Developer Commit → GitHub → Jenkins → Docker → EC2 → Application → CloudWatch → SNS → S3 Backup

Developer Commit → GitHub → Jenkins → Docker → EC2 → Application → CloudWatch → SNS → S3 Backup

17. Results

The application deployed successfully. Jenkins automated deployment, Docker container executed correctly,

CloudWatch collected metrics and S3 stored backups.

The application deployed successfully. Jenkins automated deployment, Docker container executed correctly,

CloudWatch collected metrics and S3 stored backups.

18. Future Enhancements

- HTTPS using ACM
 - Application Load Balancer
 - Auto Scaling
 - Terraform Infrastructure as Code
 - Kubernetes (EKS)
 - Prometheus and Grafana
-
- HTTPS using ACM
 - Application Load Balancer
 - Auto Scaling
 - Terraform Infrastructure as Code
 - Kubernetes (EKS)
 - Prometheus and Grafana

19. Conclusion

This project successfully demonstrates modern DevOps practices using AWS services and automation tools.

This project successfully demonstrates modern DevOps practices using AWS services and automation tools.

20. References

AWS Documentation

Docker Documentation

Jenkins Documentation

GitHub Documentation

Python Flask Documentation

AWS Documentation

Docker Documentation

Jenkins Documentation

GitHub Documentation

Python Flask Documentation

